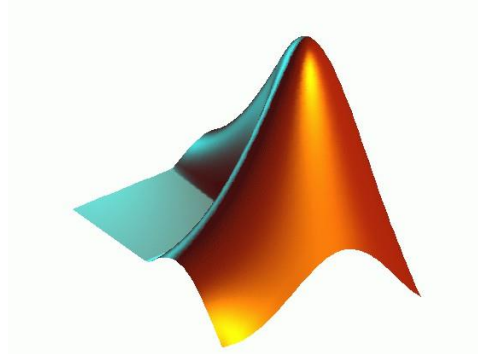


Module : Apprentissage Artificiel (Machine Learning)

Filière Parcours d'Excellence ISOC

Semestre 6

TP1 : Initiation à MATLAB et Machine Learning



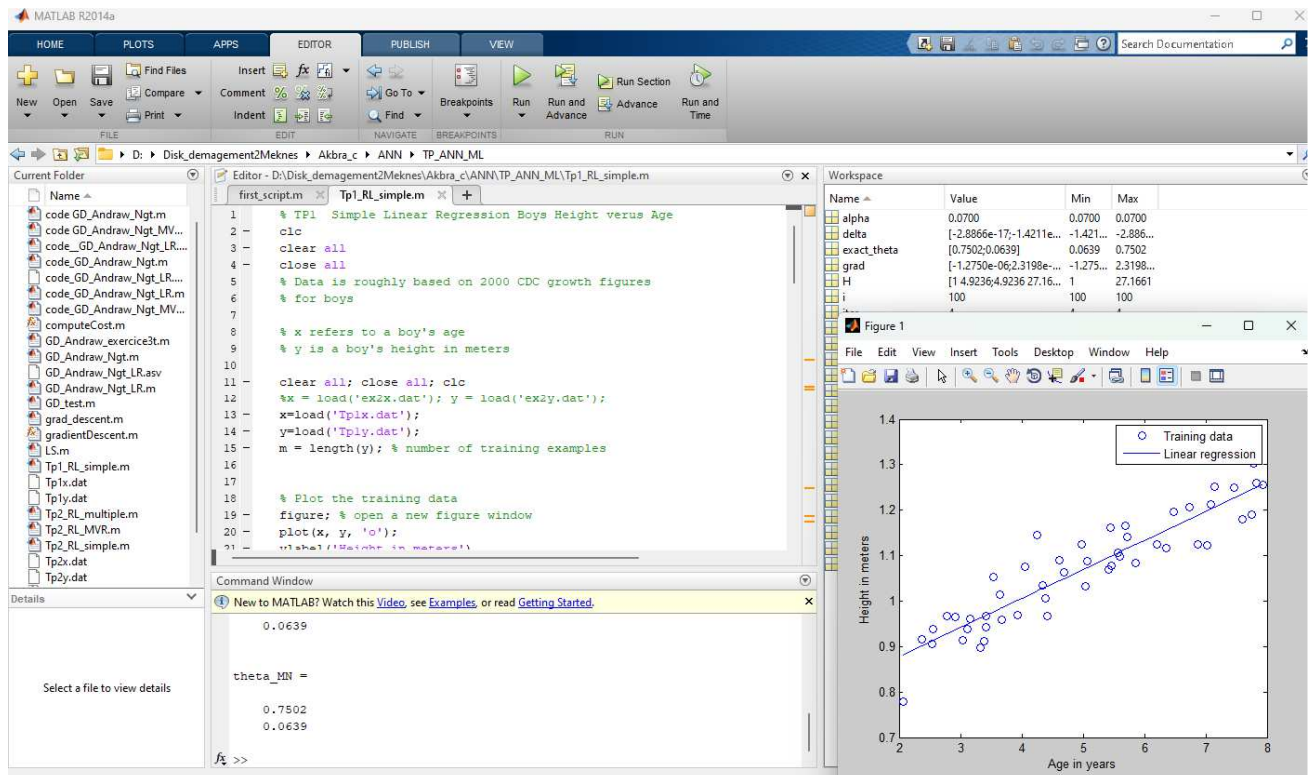
**Responsable : Brahim AKSASSE
FS Meknès
Département Informatique**

Année Universitaire : 2025-2026

1. Introduction à MATLAB

MATLAB est un logiciel de calcul numérique et de programmation largement utilisé en science des données, en traitement du signal/image et en Machine Learning, apprécié pour sa simplicité et ses nombreuses boîtes à outils spécialisées. Bien que Python domine le domaine des sciences des données, MATLAB reste très utilisé dans les milieux industriels et universitaires, notamment grâce à ses fonctionnalités avancées en optimisation, apprentissage automatique et simulation (Simulink). Ce document présente les instructions de base, les concepts de programmation et des applications pratiques de MATLAB, tout en encourageant l'auto-apprentissage à travers les ressources disponibles en ligne et en bibliothèque

2. Interface MATLAB



L'environnement MATLAB comprend :

- **Command Window**: pour exécuter des commandes directement.
- **Editor**: pour créer et exécuter des scripts et des fonctions.
- **Workspace**: pour afficher les variables en mémoire.
- **Figure Window**: pour les visualisations graphiques.
- **Current folder** : pour explorer les fichiers et les dossiers.

3. Typage

En MATLAB, on peut afficher les variables en mémoire via l'onglet **Workspace** ou avec les commandes `who` et `whos`. Le **Workspace** fournit des informations sur chaque variable, notamment son type. Contrairement à d'autres langages où les variables doivent être déclarées avec un type précis, MATLAB ajuste automatiquement le typage, ce qui peut être risqué.

La programmation en Matlab consiste en l'enchaînement d'expressions composées de variables, de nombres, d'opérateurs et de fonctions.

3.1 Variables

Il n'est pas nécessaire de déclarer le type ou la dimension des variables : Matlab choisit selon le contexte.

3.2 Constantes

Un certain nombre de fonctions prédéfinies (sans arguments) renvoient les valeurs de constantes usuelles :

pi 3.14159265358979

i unité imaginaire -1

j pareil que i

eps précision relative en virgule flottante : 2^{-52}

realmin plus petit nombre en virgule flottante : 2^{-1022}

realmax plus grand nombre en virgule flottante : 2^{+1022}

Inf Infini (résultat de 1/0 par exemple)

NaN Not A Number (résultat de 0/0 par exemple)

Attention : il est possible de réaffecter les valeurs de ces constantes (éviter d'utiliser i et j comme indice dans les boucles de calcul, notamment).

Le **typage** définit le protocole de codage d'une variable en langage machine et détermine l'espace mémoire nécessaire. Par exemple, une variable de type **logical** ne peut prendre que deux valeurs (1 pour TRUE, 0 pour FALSE) et pourrait être codée avec un seul bit en théorie. MATLAB propose plusieurs types de variables, dont :

- **double** (double précision, 64 bits),
- **logical** (binaire),
- **char** (caractère, 8 bits),
- **uint8** (entier non signé, 8 bits).

D'autres types existent et peuvent être consultés via l'aide MATLAB. Cette gestion dynamique des types simplifie l'utilisation du logiciel, mais une mauvaise manipulation peut entraîner des erreurs ou une consommation excessive de mémoire.

3.3 Les structures

Matlab permet de manipuler des données sous formes de *structures* : les données sont rangées dans une arborescence et accessibles par des *champs*. L'exemple suivant permet de comprendre comment créer une structure simple :

```
toto=[];  
toto.tata=[1 2 3 4];  
toto.titi=2;  
toto
```

toto =

```
tata: [1 2 3 4]  
titi: 2
```

La structure toto contient deux champs : *tata* (qui est un vecteur de dimension 4) et *titi* (qui est un entier). Pour accéder à un élément de la structure, on utilise le point :

```
>> toto.tata
```

ans =

```
1 2 3 4
```

4. Affichage d'une donnée

En tapant simplement dans la fenêtre de commande `a=9`, il s'affiche sur deux lignes le résultat de cette opération. Pour lire de nouveau la valeur de `a`, il suffit de taper `a` et de valider.

Il est possible d'afficher des données proprement avec la commande `disp`:

```
disp(a)
```

Remarque : Il est possible d'afficher une apostrophe ' en la doublant dans la chaîne de caractères :

```
disp('L''éléphant d''Ukraine est d''enfer !')
```

5. Vecteurs

Pour créer un vecteur, taper

V1=[1 2 3 4 5 6] s'affiche dans la fenêtre de commande. Elle affiche un vecteur ligne de 6 composantes V1(1)=1, V1(2)=2, V1(3)=3, V1(4)=4, V1(5)=5, V1(6)=6. Les indices en Matlab commencent à 1.

Taper ensuite :

V2 = V1' cette instruction affiche un vecteur colonne (le transposée d'un vecteur ligne est un vecteur colonne).

Dans une fenêtre de commande, taper successivement :

V3 = 1 :11

V4 = 1 :2 :11

V5 = 1 :.1 :3

Pour obtenir la valeur de la composante i , il suffit de taper

V5(i)

En tapant V1(end), on aura le dernier élément du vecteur V1. Une autre façon d'obtenir ce résultat est connaître la taille (longueur) du vecteur.

Les données dans MATLAB sont toujours représentées sous forme de vecteur ou de matrice. Une des fonctions essentielles est la détermination du minimum et du maximum des éléments d'un vecteur.

Dans une fenêtre de commande, taper successivement :

V = rand(6,1) création d'un vecteur aléatoire de taille 6x1(vecteur colonne)

a = min(V) la fonction min renvoie la valeur minimale.

[a,b] = min(V), ici la fonction renvoie le minimum et son indice.

La fonction rand permet de tirer au hasard une valeur entre 0 et 1.

6. Matrices

Les premières versions de MATLAB il y a une trentaine d'années étaient des bibliothèques d'inversion de matrices (l'acronyme MATLAB signifiant "Matrix Laboratory"). Le type fondamental des variables de MATLAB est le type matrice. Une matrice est un tableau à deux dimensions d'expressions de même type.

Soit M la matrice de nombres aléatoires suivante : M = rand(4,6) matrice aléatoire de taille 4x6. (4 lignes par 6 colonnes). On constate que les termes d'une même ligne sont séparés dans la déclaration par des virgules ou par des espaces. Les lignes sont séparées entre elles par des points-virgules. Raisonnons sur des matrices plus simples.

Tapez successivement les commandes suivantes :

A = [1 2 3 4; 5 6 7 8; 9 10 11 12; 13 14 15 16] une matrice 4x4, elle affiche

A =

```
1   2   3   4
5   6   7   8
9  10  11  12
13  14  15  16
```

B = A(2,2) : affecter l'élément A(2,2) à B, elle affiche

B =

6

C = A(:,2) affecter la deuxième colonne de A à la variable C (C sera donc un vecteur colonne), elle affiche

C =

```
2
6
10
14
```

D = A(1 :3,2 :3) extraire une sous matrice de A et l'affecter à une autre variable D. D sera une matrice de taille 3x2, elle affiche

D =

```
2   3
6   7
10  11
```

E = A' : la transposition matricielle, elle affiche

```
E =
    1    5    9   13
    2    6   10   14
    3    7   11   15
    4    8   12   16
```

Les opérations matricielles classiques (multiplication, puissance, division) sont directement accessibles par leur symbole normal. Dans la fenêtre de commande, taper

```
A*C
```

```
ans =
    100
    228
    356
    484
```

```
puis
```

```
D.*D
```

```
ans =
     4     9
    36    49
   100   121
```

De même,

```
A^2
```

```
ans =
    90   100   110   120
   202   228   254   280
   314   356   398   440
   426   484   542   600
```

```
puis
```

```
A.^2
```

```
ans =
     1     4     9    16
    25    36    49    64
    81   100   121   144
   169   196   225   256
```

Le plus intéressant et le plus inhabituel dans MATLAB est la surcharge de type des opérations. Généralement, en dehors des opérateurs classiques, MATLAB interprète toute opération dont les termes sont des matrices comme une opération à faire terme à terme.

Remarque : il est important de constater que les fonctions sont construites à partir de variables *discrétisées*, c'est-à-dire d'une succession de valeurs séparées par un intervalle donné appelé *pas de discrétisation*.

En posant

```
A = [-1.9, -0.2, 3.4; 5.6, 7, 2.4; -0.5 -2.2 3.3 ];
```

Il est facile, sous MATLAB, de rechercher les valeurs propres (*eigenvalues*) et vecteurs propres (*eigenvectors*) d'une matrice carrée avec la commande eig.

[V,D]=eig(A) ; retourne deux matrices V et D. Les colonnes de C sont les vecteurs propres et les éléments diagonaux de D sont les valeurs propres correspondantes.

```
A = [-1.9, -0.2, 3.4 ; 5.6, 7, 2.4 ; -0.5 -2.2 3.3 ]
[V,D]=eig(A)
```

```
A =
-1.9000 -0.2000  3.4000
```

```

5.6000  7.0000  2.4000
-0.5000 -2.2000  3.3000
V =
0.8627 + 0.0000i  0.0993 - 0.2231i  0.0993 + 0.2231i
-0.4919 + 0.0000i -0.8179 + 0.0000i -0.8179 + 0.0000i
-0.1173 + 0.0000i  0.3393 - 0.3954i  0.3393 + 0.3954i

```

```

D =
-2.2483 + 0.0000i  0.0000 + 0.0000i  0.0000 + 0.0000i
0.0000 + 0.0000i  5.3242 + 2.6878i  0.0000 + 0.0000i
0.0000 + 0.0000i  0.0000 + 0.0000i  5.3242 - 2.6878i

```

i ici est l'imaginaire pure ($i^2=-1$). Les valeurs peuvent être des réelles ou des complexes.

Il existe plusieurs instructions permettant de construire des matrices particulières.

Il existe également plusieurs instructions relatives à des fonctions d'analyse linéaire.

7. Commandes de base

Quelques commandes utiles :

`clc;` % Effacer la console

`clear;` % Supprimer les variables en mémoire

`close all;` % Fermer toutes les figures

Manipulation des variables et des matrices

`A = [1 2 3; 4 5 6; 7 8 9];` % Création d'une matrice 3x3

`B = rand(3,3);` % Matrice aléatoire 3x3

`C = A + B;` % Opérations matricielles

`v = A(:,2);` % Extraction de la 2ème colonne

8. Manipulation des données

8.1 Statistiques de base

```

X = randn(1,100); % Génération de 100 valeurs aléatoires
mean_X = mean(X) % Moyenne
std_X = std(X) % Écart-type

```

mean_X =

0.0548

std_X =

0.8566

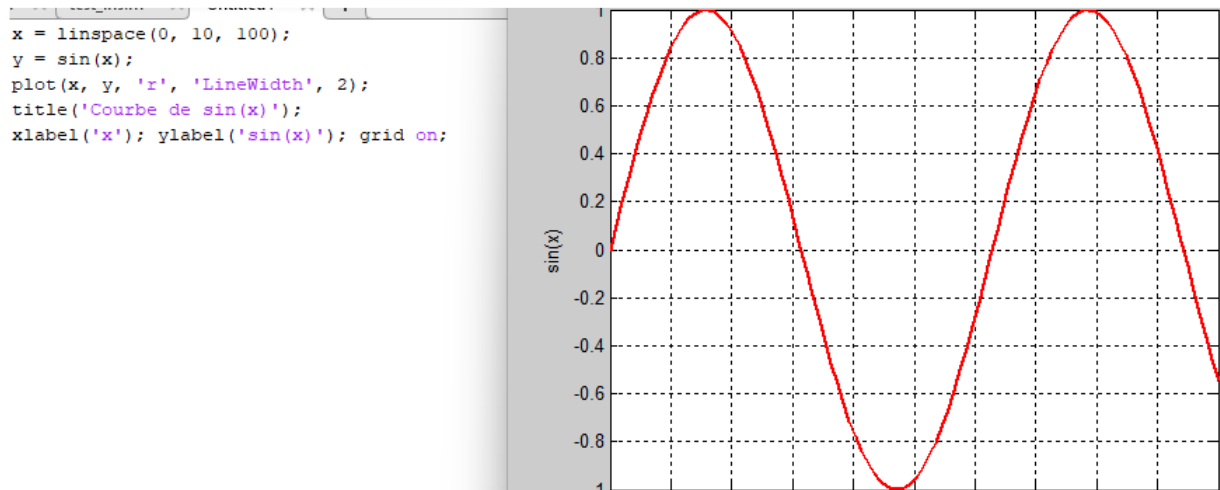
8.2 Chargement et sauvegarde de données

`data = load('data.txt');` % Charger un fichier texte

`save('result.mat', 'X');` % Sauvegarder une variable

9. Visualisation des données

9.1 Courbe simple



9.2 Ajouter des courbes à une figure existante

La commande `hold on` permet de conserver la figure courante : le tracé suivant ne se fait pas dans une nouvelle fenêtre mais se superpose au tracé précédent (en adaptant éventuellement les échelles).

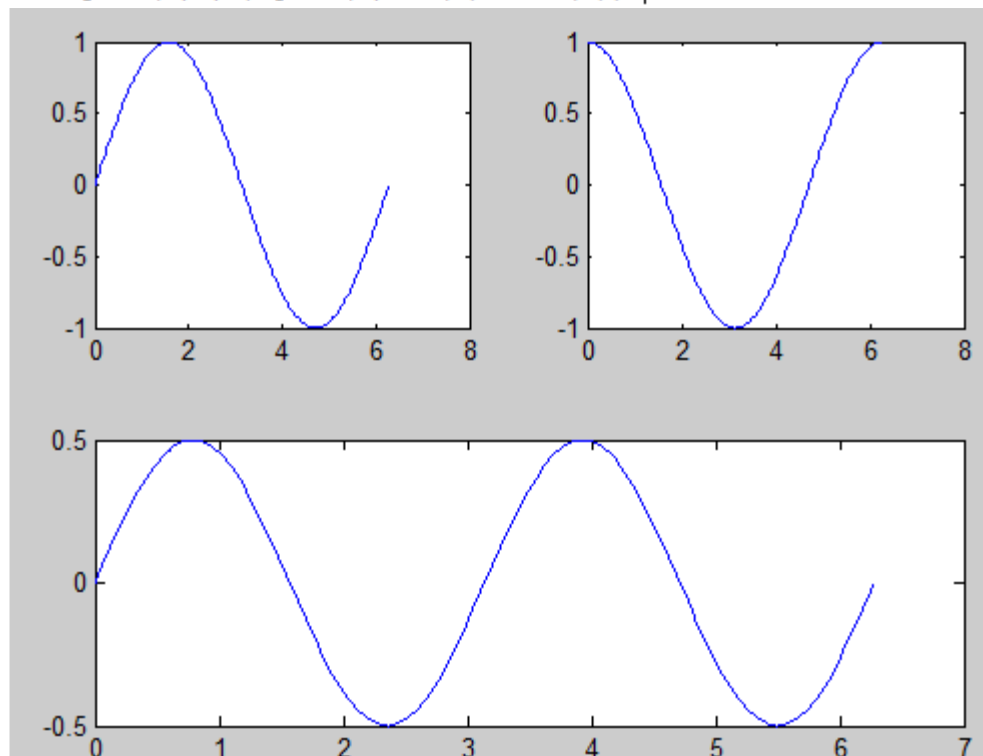
9.3 Plusieurs courbes dans une même fenêtre

La commande `subplot` permet de partitionner la fenêtre de la figure et de tracer plusieurs courbes. Considérons l'exemple suivant

```

x=0:pi/100:2*pi;
subplot(2,2,1);plot(x,sin(x));
subplot(2,2,2);plot(x,cos(x));
subplot(2,1,2);plot(x,sin(x).*cos(x));

```



Le premier subplot partitionne en deux lignes et deux colonnes et place le pointeur sur la première case. Le second subplot partitionne de la même manière et place le pointeur sur la deuxième case. Le troisième subplot partitionne en deux lignes et une seule colonne et place le pointeur sur la deuxième case (la deuxième ligne).

Une fois le graphique créé, il est possible de rajouter des titres, des commentaires, etc... en utilisant la fonction `Tools / Edit Plot` de la barre de menu de la fenêtre graphique. De manière plus générale, et cela sort du cadre de ce manuel, les données décrivant l'ensemble d'un graphique sont stockées sous la forme d'une *structure*. Il est donc possible de pointer directement sur un élément d'un

graphique (axe, titre, donnée, etc...) et de le modifier en utilisant des fonctions spécialisées. Pour plus de détails consulter l'aide de Matlab.

9.4 Placer des titres et des commentaires

Les commandes xlabel, ylabel, title, text permettent de commenter les axes x, y, de placer un titre au graphique ou de placer un commentaire n'importe où sur le graphique.

9.5 Histogramme et nuage de points

```
data = randn(1000,1);
```

```
hist (data, 30);% retourne la fréquence d'apparition de chaque valeur (nombre de fois qu'elle apparaît dans cet échantillon). L'intervalle [min(data), max(data)] sera divisé en 30 bins.
```

```
x = rand(50,1); y = rand(50,1);
```

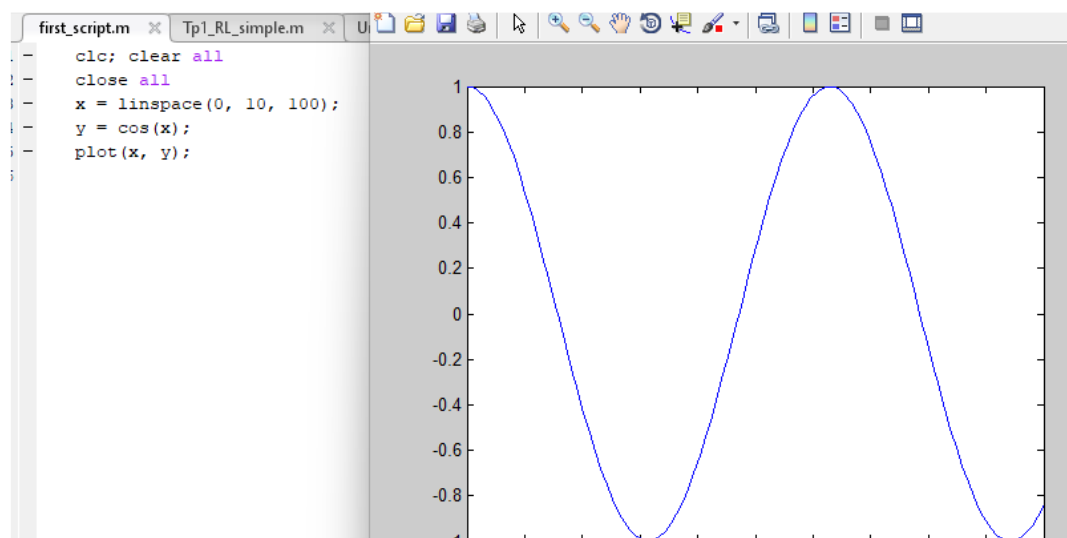
```
scatter(x, y, 'filled');% trace le nuage de point
```

10. Scripts et fonctions MATLAB

10.1 Création de scripts

Un script est un fichier.m contenant des commandes MATLAB.

% Mon premier script



10.2 Création de fonctions

```
care.m  +
function y = carre(x)
    y = x.^2;
end
```

Appel de la fonction :

```
>> result = care(5)
```

```
result =
    25V
```

10.3 Instructions et structures de contrôle

Matlab dispose d'un certain nombre d'instructions de contrôle :

- if
- switch
- for
- while
- continue
- break

Pour la syntaxe exacte et pour des exemples, utiliser l'aide en ligne : help if (par exemple).

11. Introduction au Machine Learning avec MATLAB

11.1 Chargement de données

```
load iris_dataset % Chargement du jeu de données iris
% Size of the dataset
[numFeatures, numSamples] = size(irisInputs);
fprintf('Number of features: %d\n', numFeatures);
fprintf('Number of samples : %d\n', numSamples);
```

Number of features: 4

Number of samples : 150

11.2 Exploration des données

```
% Convert one-hot targets to class indices
[~, classIndices] = max(irisTargets, [], 1);
% Count how many samples per class
classCounts = histcounts(classIndices, 1:4); % 3 classes: 1, 2, 3
% Display
classNames = {'Setosa', 'Versicolor', 'Virginica'};
for i = 1:length(classCounts)
    fprintf('%s: %d samples\n', classNames{i}, classCounts(i));
end
```

Setosa: 50 samples

Versicolor: 50 samples

Virginica: 50 samples

```
featureNames = {'SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth'};
% Compute stats
featureMeans = mean(irisInputs, 2);
featureStds = std(irisInputs, 0, 2);
featureMins = min(irisInputs, [], 2);
featureMaxs = max(irisInputs, [], 2);

% Display table
fprintf('\nFeature Statistics:\n');
fprintf('Feature\t\tMean\tStd\tMin\tMax\n');
for i = 1:numFeatures
    fprintf('%s\t%.2f\t%.2f\t%.2f\t%.2f\n', ...
        featureNames{i}, featureMeans(i), featureStds(i), featureMins(i), featureMaxs(i));
end
```

Feature Statistics:

Feature	Mean	Std	Min	Max
SepalLength	5.84	0.83	4.30	7.90
SepalWidth	3.05	0.43	2.00	4.40
PetalLength	3.76	1.76	1.00	6.90
PetalWidth	0.76	0.10	2.50	

11.3 Application d'un modèle de classification simple

```

% Transpose inputs to samples x features
X = irisInputs'; % Now 150x4
[~, Y] = max(irisTargets, [], 1); % Convert one-hot to class labels (1,2,3)

% Create the KNN model (default K=1)
knnModel = fitcknn(X, Y, 'NumNeighbors', 3);
% Predict on training data
Y_pred = predict(knnModel, X);

% Compute accuracy
accuracy = sum(Y_pred == Y) / numel(Y);
fprintf('KNN Classification Accuracy: %.2f%%\n', accuracy * 100);
confMat = confusionmat(Y, Y_pred);
disp('Confusion Matrix:');
disp(confMat);

```

12. Conclusion

Ce support a introduit MATLAB et ses bases indispensables pour le Machine Learning. Les prochaines étapes incluront l'implémentation de modèles plus avancés comme la régression, la classification et les réseaux neuronaux.